

TensorFlow2



TensorFlow

製作人: 甯文駿

以下文字敘述都是由 Gemini 產生，可搭配程式碼一同觀看。

第 09 章_下載 MNIST 手寫數字資料集.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC09%E7%AB%A0_%E4%B8%8B%E8%BC%89MNIST%E6%89%8B%E5%AF%AB%E6%95%B8%E5%AD%97%E8%B3%87%E6%96%99%E9%9B%86.ipynb

以下是該檔案內容的重點介紹：

1. 資料集概況

- **主題：**MNIST (Modified National Institute of Standards and Technology) 包含了 0 到 9 的手寫數字影像。
- **資料量：**
 - **訓練集 (Training Set)：**60,000 筆影像與標籤，用於讓模型學習。
 - **測試集 (Test Set)：**10,000 筆影像與標籤，用於評估模型的準確度。
- **影像規格：**每張影像的大小為 28x28 像素。

2. 資料結構

- **影像資料 (X)：**以 `numpy.ndarray` 儲存，每個像素的值介於 0 到 255 之間（代表灰階深淺，0 為黑色/背景，255 為白色/筆畫）。
- **標籤資料 (y)：**對應影像的正確數字。例如，第一筆資料（索引為 0）的標籤是 5。

3. 主要程式步驟

- **載入資料：**使用 `mnist.load_data()` 自動從雲端下載並切割好訓練與測試資料。
- **維度確認：**透過 `.shape` 指令確認資料的筆數與影像大小。

- **視覺化定義**：建立 `plot_image` 函數，使用 `matplotlib` 庫將矩陣數據轉換回黑白影像，方便人類觀察。
- **批量展示**：定義了 `plot_images_labels_prediction` 函數，可以一次顯示多張圖片及其對應的標籤（Label），甚至預留了顯示模型預測結果（Prediction）的位置。

4. 實例展示

在檔案最後，程式展示了訓練集中的前 10 筆資料。可以看到第一張圖是一個手寫的 5，對應的標籤也是 5，這證明了資料與標籤是正確對應的。

第 11 章_MLP 辨識 MNIST 手寫數字.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC11%E7%AB%A0_MLP%E8%BE%A8%E8%AD%98MNIST%E6%89%8B%E5%AF%AB%E6%95%B8%E5%AD%97.ipynb

以下是該檔案內容的重點介紹：

1. 資料預處理 (Data Preprocessing)

為了讓神經網路能有效學習，檔案執行了一個關鍵步驟：

- **維度轉換 (Reshape)**：將原始 28×28 像素的二維影像「攤平」成 784 維的一維向量（因為 MLP 的輸入層是一維的）。
- **標準化 (Normalization)**：將 0 到 255 的像素值除以 255，轉換成 0.0 到 1.0 之間的數值，這有助於模型收斂。
- **標籤轉換 (One-hot Encoding)**：將數字標籤（如 5）轉換成 10 維的向量（如 $[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]$ ），方便計算損失函數。

2. 建立 MLP 模型 (Model Architecture)

檔案使用 Keras 的 Sequential 建立了一個深層的神經網路：

- **輸入層**：接收 784 個神經元。
- **隱藏層 1**：1000 個神經元，使用 ReLU 激活函數。
- **Dropout 層**：設定為 0.5，隨機丟棄一半的神經元以防止過擬合 (Overfitting)。
- **隱藏層 2**：同樣為 1000 個神經元加 Dropout。
- **輸出層**：10 個神經元，使用 Softmax 激活函數來輸出 10 個數字的機率分佈。

3. 模型訓練 (Training)

- 使用 Adam 優化器與 Categorical Crossentropy 損失函數。

- 設定訓練 10 次 (Epochs)，每批次處理 200 筆資料。
- 訓練過程中，從訓練集撥出 20% 作為**驗證集 (Validation Set)**。

4. 訓練結果與視覺化 (Visualization)

- 檔案中定義了 `show_train_history` 函數，繪製出準確度 (Accuracy) 與損失 (Loss) 隨 Epoch 增加的曲線圖。
- 結果顯示訓練集準確度最終達到約 **98.25%**，驗證集約為 **97.91%**。

5. 模型評估與測試 (Evaluation)

- **測試準確度**：模型在完全沒看過的 10,000 筆測試集上，準確度達到 **98.2%**。
- **機率分析**：展示了模型如何判斷一張圖。例如數字 7 的預測機率高達 0.999992。
- **混淆矩陣 (Confusion Matrix)**：利用 `pandas.crosstab` 呈現模型在哪些數字上容易搞混。結果顯示測試集中總共僅猜錯了 **180 張** 圖片。

第 15 章_CNN 辨識 MNIST 手寫數字.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC15%E7%AB%A0_CNN%E8%BE%A8%E8%AD%98MNIST%E6%89%8B%E5%AF%AB%E6%95%B8%E5%AD%97.ipynb

以下是該檔案內容的重點介紹：

1. 資料預處理 (Data Preprocessing)

為了符合 CNN 的輸入格式，資料進行了以下調整：

- **維度轉換 (Reshape)**：將影像轉換為四維張量（樣本數，28，28，1）。最後一個維度 1 代表單色（灰階）通道。
- **標準化**：將像素值除以 255，縮放至 0.0 到 1.0 之間。
- **標籤編碼**：使用 `to_categorical` 進行 One-hot 編碼。

2. 建立 CNN 模型架構

檔案中使用 Keras 建立了典型的 CNN 結構：

- **卷積層 (Convolution Layer)**：
 - 第一層使用 32 個 5×5 的濾波器 (Filters)。
 - 第二層增加到 64 個 5×5 的濾波器。
 - 兩層均使用 ReLU 激活函數，並設定 `padding='same'` 以保持影像尺寸。
- **池化層 (Pooling Layer)**：在每層卷積後接續 `MaxPooling2D`（尺寸 2×2 ），用來下採樣並減少參數運算量。
- **全連接層**：
 - **平坦層 (Flatten)**：將二維特徵圖拉直為一維向量。
 - **隱藏層**：包含 128 個神經元。

- **輸出層**：10 個神經元，使用 **Softmax** 激活函數輸出 0~9 的機率。

3. 模型訓練與效能

- 使用 **Adam** 優化器與 **Categorical Crossentropy** 損失函數進行編譯。
- **訓練過程**：設定 **Epochs** 為 10 次，批次大小為 200。
- **效能表現**：經過 10 次訓練後，**驗證集準確度 (val_acc)** 達到了約 99.14%，優於傳統 **MLP** 的表現。

4. 硬體加速資訊

檔案最後包含了一個檢查硬體加速器的函數：

- **GPU 使用**：確認在 **Google Colab** 環境中使用了 **Tesla T4 GPU** 進行運算加速。
- **環境資訊**：顯示了 **NVIDIA** 驅動版本與 **CUDA** 版本等系統詳細資訊。

第 17 章_視覺化 MNIST CNN 特徵圖.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC17%E7%AB%A0_%E8%A6%96%E8%A6%BA%E5%8C%96MNIST%20CNN%E7%89%B9%E5%BE%B5%E5%9C%96.ipynb

以下是該檔案內容的重點介紹：

1. 核心目標：視覺化特徵圖 (Feature Maps)

在 CNN 中，卷積層會利用多個濾波器 (Filters) 來提取影像特徵 (如線條、邊緣、形狀)。本章透過程式碼將這些隱藏在網路內部的特徵圖繪製出來，讓使用者直觀理解神經網路的特徵提取過程。

2. 關鍵實作步驟

根據檔案內容，視覺化的流程如下：

- **載入已訓練模型：**通常接著第 15 章訓練好的 CNN 模型繼續執行。
- **建立「中間模型」：**
 - 使用 Keras 的 Model 類別，重新定義一個輸出為「特定卷積層」的模型。
 - 例如：輸入一張圖片，輸出則是第一層卷積層 (conv2d_2) 或第二層卷積層 (conv2d_3) 產生的結果。
- **輸入測試影像：**將一張手寫數字圖片 (如數字 7) 輸入這個中間模型。
- **繪製特徵圖：**
 - **第一層卷積層：**可以看到 32 個濾波器分別提取了不同角度的邊緣或輪廓。
 - **第二層卷積層：**隨著層數加深，特徵圖會變得更加抽象且難以辨認，這代表模型正在整合更複雜的空間特徵。

3. 特徵提取的演變

檔案中展示了影像通過不同層級的變化：

- **卷積層 1 (Conv2D_1)**：特徵圖保留了大部分數字的原始輪廓，但會強調不同的邊界（如垂直線或水平線）。
- **池化層 1 (MaxPooling2D_1)**：影像尺寸縮小一半（從 28 x 28 變為 14 x 14），但關鍵特徵被保留。
- **卷積層 2 (Conv2D_2)**：產生的 64 個特徵圖更加破碎、細碎，這反映了深層網路負責處理高階且局部細節的特性。

第 18 章_下載 Cifar_10 影像辨識資料集.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC18%E7%AB%A0_%E4%B8%8B%E8%BC%89Cifar_10%E5%BD%B1%E5%83%8F%E8%BE%A8%E8%AD%98%E8%B3%87%E6%96%99%E9%9B%86.ipynb

以下是該檔案內容的重點介紹：

1. 什麼是 CIFAR-10 資料集？

CIFAR-10 是一個包含 60,000 張 32 x 32 像素彩色圖片的資料集。這些圖片被分為 10 個不同的類別，每類有 6,000 張圖：

- **載具類**：飛機、汽車、船、卡車。
- **動物類**：鳥、貓、鹿、狗、青蛙、馬。

2. 資料載入與規格

- **載入方式**：使用 `keras.datasets.cifar10.load_data()` 直接從雲端下載。
- **資料切割**：
 - **訓練集**：50,000 筆圖片與標籤。
 - **測試集**：10,000 筆圖片與標籤。
- **影像結構**：每張圖片的維度為 (32, 32, 3)，最後的 3 代表 RGB 三原色通道。

3. 資料預處理 (Preprocessing)

為了讓神經網路（特別是 CNN）有更好的表現，檔案中執行了以下處理：

- **影像標準化**：將 0 到 255 的像素值轉換為 `float32` 類型，並除以 255，使其落於 0.0 到 1.0 之間。
- **標籤 One-hot 編碼**：使用 `to_categorical` 將數字類別 (0-9) 轉換成 10 維的向量。例如，標籤 3 (貓) 會變成 [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]。

0, 0]。

4. 影像視覺化

檔案中定義了 `plot_images_labels_prediction` 函數：

- **功能**：可以一次顯示多張彩色圖片，並在圖片上方標註其類別名稱（如 "airplane", "bird" 等）。
- **目的**：讓使用者在開始訓練模型前，先直觀觀察資料的特徵與品質。

5. 與 MNIST 的主要差異

這份檔案強調了 CIFAR-10 比起 MNIST 手寫數字更具挑戰性的原因：

- **色彩**：從灰階變為彩色，運算量增加三倍。
- **複雜度**：實體物件受背景、光照、角度影響較大，需要更強大的卷積神經網路（CNN）架構來處理。

第 20 章_CNN 辨識 Cifar_10 影像.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC20%E7%AB%A0_CNN%E8%BE%A8%E8%AD%98Cifar_10%E5%BD%B1%E5%83%8F.ipynb

以下是該檔案內容的重點介紹：

1. 資料預處理 (Data Preprocessing)

- **格式轉換**：將 CIFAR-10 的彩色圖片轉換為 float32 格式。
- **標準化 (Normalization)**：將像素值除以 255，使其落於 0 到 1 之間，有助於提高模型訓練效率。
- **標籤編碼**：將 10 種物品類別的標籤進行 One-hot Encoding 轉換。

2. CNN 模型架構設計

為了處理彩色的實體影像，本章建立了比之前更深層的網路架構：

- **多層卷積與池化**：
 - **卷積層 1**：使用 32 個濾波器，並搭配 Dropout (0.25) 隨機丟棄神經元以防止過擬合。
 - **池化層 1**：進行縮減影像尺寸的下採樣。
 - **卷積層 2**：增加至 64 個濾波器，進一步提取更複雜的特徵。
 - **池化層 2**：再次縮減尺寸。
- **全連接層**：
 - **平坦層 (Flatten)**：將特徵圖轉換為一維向量。
 - **隱藏層**：包含 1024 個神經元，並再次使用 Dropout (0.25) 強化泛化能力。
 - **輸出層**：10 個神經元，使用 Softmax 激活函數輸出 10 類物品的預測機率。

3. 模型訓練 (Training)

- **訓練配置**：使用 Adam 優化器與 Categorical Crossentropy 損失函數。
- **訓練執行**：設定訓練 10 次 (Epochs)，並從中撥出 20% 資料作為驗證集。
- **準確度表現**：經過 10 次訓練後，驗證集準確度大約達到 73% 到 74% 左右。這反映出彩色影像辨識的難度遠高於手寫數字。

4. 預測與評估

- **實際預測**：展示了模型對測試集圖片進行分類的結果，例如辨識出飛機、貓或狗。
- **機率分佈**：可以觀察模型對於特定圖片判斷為各種類別的信心百分比。

第 21 章_視覺化 Cifar_10 影像_CNN 特徵圖.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC21%E7%AB%A0_%E8%A6%96%E8%A6%BA%E5%8C%96Cifar_10%E5%BD%B1%E5%83%8F_CNN%E7%89%B9%E5%BE%B5%E5%9C%96.ipynb

與第 17 章（MNIST 視覺化）相比，本章展示了彩色通道與更深層架構下的特徵提取過程。

1. 核心目標：理解彩色影像的特徵提取

彩色影像包含 RGB 三個通道，本章透過提取中間層的輸出，讓使用者觀察濾波器（Filters）如何從色彩資訊中辨識出物體的輪廓、紋理與局部特徵。

2. 關鍵實作內容

根據檔案內容，視覺化的流程與發現如下：

- **建立中間輸出模型：**重新定義一個以原始 CNN 為基礎的模型，將其輸出指向特定的卷積層（如 conv2d 或 conv2d_1）。
- **輸入彩色測試影像：**選取一張 CIFAR-10 的圖片（例如「鳥」或「飛機」）作為輸入。
- **第一層卷積視覺化：**
 - 展示了 32 個特徵圖。由於是網路的最前端，影像仍保留明顯的色彩邊界與物體形狀。
 - 不同的濾波器會對不同的顏色通道（紅、綠、藍）或特定方向的邊緣產生反應。
- **深層卷積視覺化：**
 - 展示了更後端的 64 個特徵圖。此時影像已經高度抽象化，變成點狀或線段的組合。
 - 這代表模型已經將原始像素轉換為「高階特徵」，例如辨識出「翅膀的線條」或「車輪的圓弧」，而非單純的顏色。

3. 本章的學習重點

- **黑盒子透明化**：藉由觀察特徵圖，可以確認模型是否真的抓取到了物體的關鍵特徵（例如辨識貓時，是否抓取到了耳朵或鬍鬚的特徵）。
- **空間資訊的壓縮**：觀察池化層（Pooling）如何縮減影像尺寸，但同時保留具代表性的資訊。
- **彩色 vs. 灰階**：理解彩色影像在第一層卷積時，會比灰階影像產生更豐富、多樣的特徵反應。

第 24 章_EarlyStopping.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC24%E7%AB%A0_EarlyStopping.ipynb

以下是該檔案內容的重點介紹：

1. 核心目標：解決過擬合與節省資源

- **防止過擬合**：在訓練過程中，模型在訓練集上的準確度會持續上升，但驗證集的準確度通常會在某一點後開始下降。早停法能在驗證集表現不再進步時自動停止訓練，確保模型保有最佳的泛化能力。
- **節省時間**：避免不必要的運算，當模型已經不再進步時，不需要跑完預設的所有訓練次數 (Epochs)。

2. 關鍵實作：Keras Callback

檔案中使用了 `tensorflow.keras.callbacks.EarlyStopping` 來達成此功能，主要參數包括：

- **monitor**：監控的指標，通常設定為 `val_loss` (驗證集損失) 或 `val_acc` (驗證集準確度)。
- **patience**：容忍度。代表如果連續幾個 Epochs 指標都沒有進步，才真正停止訓練。例如設定為 5，表示指標連續 5 次沒進步就停下來。
- **verbose**：訊息層級。設定為 1 時，當訓練被強制停止，系統會印出提示訊息。
- **mode**：模式。若監控指標是 Loss 則選 `min` (越小越好)；若是 Accuracy 則選 `max` (越大越好)。

3. 實作流程

根據檔案內容，其操作步驟如下：

- **定義 Callback**：先設定好 `EarlyStopping` 物件的參數。
- **加入訓練**：在呼叫 `model.fit()` 時，將此物件傳入 `callbacks` 參數

中。

- **觀察結果：**檔案展示了原本預設要跑較多次的訓練，因為觸發了早停機制，在中間某個 Epoch 就提早結束，並保留了表現較好的權重。

第 25 章_ImageDataAugment.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC25%E7%AB%A0_ImageDataAugment.ipynb

以下是該檔案內容的重點介紹：

1. 核心目標：小樣本也能練出好模型

- **擴充資料量**：當手邊的原始圖片不足時，資料增強可以產生大量的變體。
- **提升穩定性**：讓模型學習辨識不同角度、位置或光澤下的物體，使其在現實世界（非實驗室環境）的辨識率更穩定。

2. 關鍵實作：ImageDataGenerator

檔案中使用了 Keras 的 ImageDataGenerator 類別，透過以下參數對影像進行即時變換：

- **rotation_range**: 隨機旋轉角度。
- **width_shift_range** / **height_shift_range**: 水平或垂直平移。
- **shear_range**: 隨機傾斜（剪切）變換。
- **zoom_range**: 隨機縮放影像。
- **horizontal_flip**: 隨機水平翻轉。
- **fill_mode**: 旋轉或平移後產生的空隙要如何填補。

3. 實作流程與視覺化

- **動態產生**：資料增強是在訓練過程中「一邊跑一邊產生」變體，不會佔用大量硬碟空間，且模型每一輪看到的圖片都會略有不同。
- **效果展示**：檔案中特別展示了將一張「狗」或「貓」的原始圖片，透過增強後變成 10~20 張不同姿勢、位置的圖片，讓使用者直觀理解增強的效果。

第 26 章_BatchNormalization.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC26%E7%AB%A0_BatchNormalization.ipynb

以下是該檔案內容的重點介紹：

1. 核心目標：解決內部協變量偏移

- **加速收斂**：在深層網路中，每一層的輸入分佈會隨著前一層參數的更新而改變，導致學習速度變慢。BN 透過將每一層的神經元輸出重新標準化（平均值為 0，變異數為 1），讓模型可以使用更高的學習率（Learning Rate）來快速訓練。
- **提高穩定性**：BN 具有類似正規化（Regularization）的效果，能減輕模型對權重初始化的依賴，並降低過擬合（Overfitting）的風險。

2. 關鍵實作：BatchNormalization 層

檔案中使用了 `tensorflow.keras.layers.BatchNormalization`，通常放置在卷積層（Conv2D）或全連接層（Dense）之後，激活函數（Activation）之前或之後。其運作邏輯如下：

- **計算批次統計量**：針對每一個訓練小批次（Mini-batch），計算其平均值與標準差。
- **標準化與縮放**：將數據標準化後，再加上兩個可學習的參數 γ （縮放）與 β （偏移），讓網路能自行決定是否要保留標準化的結果。

3. 實作流程與對比

根據檔案內容，其操作步驟如下：

- **模型建構**：在 CNN 的卷積層之後加入 `BatchNormalization()` 層。
- **效能對比**：檔案通常會展示加入 BN 前後的訓練曲線。觀察結果可以發現，加入 BN 的模型其 Loss 下降得更快，且 Validation Accuracy

(驗證準確度) 更早達到高點。

第 27 章_Functional_API 建立模型.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC27%E7%AB%A0_Functional_API%E5%BB%BA%E7%AB%8B%E6%A8%A1%E5%9E%8B.ipynb

以下是該檔案內容的重點介紹：

1. 核心目標：擺脫線性堆疊的限制

- **Sequential 的侷限**：Sequential 只能建構單一輸入、單一輸出且層層堆疊的線性網路。
- **Functional API 的優勢**：可以建構非線性拓樸的網路，例如具有**多個輸入、多個輸出**，或是層與層之間有**共享層、分支結構與殘差連接**（Residual Connection）的複雜模型。

2. 關鍵語法差異

在 Functional API 中，每一層都被視為一個「函數」，您需要明確定義輸入張量並將其傳遞給下一層：

- **定義輸入層**：使用 `input_tensor = Input(shape=(784,))`。
- **層的連接**：每一層後面接著括號來接收上一層的輸出，例如 `x = Dense(64, activation='relu')(input_tensor)`。
- **定義模型**：最後使用 `model = Model(inputs=input_tensor, outputs=output_tensor)` 將輸入與輸出串接起來。

3. 實作內容：以 MNIST 為例

檔案中透過 Functional API 重新實作了 MNIST 手寫數字辨識模型：

- **模型建構**：展示了如何用函數式語法建立隱藏層與輸出層，並成功編譯模型。
- **訓練與評估**：雖然建構方式不同，但訓練（fit）與評估（evaluate）的流程與 Sequential 完全相同。
- **模型視覺化**：檔案中通常會使用 `plot_model` 函數來繪製模型圖，讓使

用者能清晰看到 Functional API 建立出來的層級邏輯。

第 28 章_Model_Subclassing_API 建立模型.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC28%E7%AB%A0_Model_Subclassing_API%E5%BB%BA%E7%AB%8B%E6%A8%A1%E5%9E%8B.ipynb

以下是該檔案內容的重點介紹：

1. 核心目標：物件導向的模型設計

- **高度自定義**：與 Sequential 或 Functional API 不同，Model Subclassing 允許您透過繼承 Keras 的 Model 類別，完全掌控模型的「正向傳播 (Forward Pass)」邏輯。
- **物件導向**：將模型視為一個 Python 物件，讓結構更易於模組化與重複使用。

2. 關鍵實作架構

在 Model Subclassing 中，模型主要由兩個部分組成：

- **__init__() 建構子**：在此定義模型所需的所有層 (Layers)。例如：
`self.dense1 = layers.Dense(64)`。這些層會在模型初始化時被建立。
- **call() 方法**：在此定義數據如何流過這些層 (即計算邏輯)。這讓您可以使用 Python 的原生語法 (如 if 判斷式或 for 迴圈) 來控制模型的運算流程。

3. 實作流程：以 MNIST 為例

檔案中示範了如何透過子類化實作手寫數字辨識模型：

- **定義類別**：建立一個繼承自 `tf.keras.Model` 的類別。
- **層的實例化**：在 `__init__` 中宣告卷積層、平坦層與全連接層。
- **邏輯串接**：在 `call` 函數中依序呼叫這些層，並回傳結果。
- **編譯與訓練**：建立該類別的實例後，後續的 `compile` 與 `fit` 流程與前幾章完全一致。

第 31 章_自訂程式碼片段.ipynb

連結：

https://github.com/ningmao47/TensorFlow2/blob/main/%E7%AC%AC31%E7%AB%A0_%E8%87%AA%E8%A8%82%E7%A8%8B%E5%BC%8F%E7%A2%BC%E7%89%87%E6%AE%B5.ipynb

以下是該檔案內容的重點介紹：

1. 核心目標：打造個人專屬的程式庫

- **提升效率**：在深度學習開發中，許多預處理、模型架構或繪圖函數會重複使用。透過自訂程式碼片段，不需要每次重新編寫或翻找舊檔案。
- **模組化管理**：將繁瑣的設定（如硬體檢查、資料載入、混淆矩陣繪製）儲存為片段，方便在不同的 Notebook 之間快速套用。

2. 關鍵實作流程

根據檔案內容，設定自訂程式碼片段的步驟如下：

- **建立來源 Notebook**：建立一個專門存放常用程式碼的 Colab 檔案。
- **程式碼組織**：在該檔案中，每一個 Markdown 標題（#）代表程式碼片段的名稱，其下方的第一個程式碼儲存格（Code Cell）即為該片段的內容。
- **設定連結**：
 1. 複製該來源 Notebook 的網址（URL）。
 2. 在 Colab 的「工具」→「設定」→「網站」中，將網址貼入「自訂程式碼片段筆記本網址」欄位。
- **調用片段**：設定完成後，在左側工具欄的「程式碼片段」分頁中即可搜尋並插入預設好的程式碼。

3. 檔案中的示範範例

檔案中展示了幾種適合製作成片段的內容：

- **硬體檢查**：快速確認目前環境是使用 GPU 或 TPU。
- **視覺化函數**：例如前幾章常用的 `show_train_history` 或 `plot_images_labels_prediction`。
- **模型評估工具**：如混淆矩陣的計算與繪製。

4. 學習價值

這章內容代表您從「學習演算法」進階到了「優化開發流程」的階段：

- **工程化思維**：學會如何管理自己的開發資產，減少重複性勞動。